Comprehensive Viva & Evaluator Q&A Guide

This guide compiles essential oral examination and evaluation questions for the **Smart Pill Box** embedded project.

---

1. General Embedded & Architecture Questions

Q1: Can you explain the overall system block diagram of your project?

> ***Answer:***
>
> *The project is centered on the **STM32F103C8T6 (ARM Cortex-M3)** microcontroller. It connects to two slave devices on a shared **I2C1 bus (PB6/PB7)**: a 0.96-inch **SSD1306 128x64 OLED display** and a **DS1307 Real-Time Clock module**.*
>
> *User interaction is driven by **three pushbuttons (PB12, PB14, PB15)** configured with internal pull-ups. Outputs include **two indicator LEDs (PB1, PB10)** for pill differentiation and a **piezo buzzer (PB0)** for acoustic alerting. Code flashing and hardware debugging are handled via **ST-LINK v2** over **Serial Wire Debug (SWD)** pins PA13 and PA14.*

Q2: Why is the STM32F103C8T6 preferred over an 8-bit Arduino (ATmega328P)?

> ***Answer:***
>
> *1. **Processing Power**: 72 MHz 32-bit Cortex-M3 vs 16 MHz 8-bit AVR.*
>
> *2. **Memory**: 20 KB SRAM (enough for a 1 KB display framebuffer + rich application logic) vs 2 KB on Uno.*
>
> *3. **Hardware Debugging**: Built-in SWD hardware breakpoint unit and trace vs basic serial printing on Arduino.*
>
> *4. **Industrial Architecture**: Direct register access, DMA, nested interrupt vector controller (NVIC), and lower power modes.*

---

2. Real-Time Clock (DS1307) & Bus Architecture

Q3: How do the OLED and DS1307 share the same I2C lines without colliding?

> ***Answer:***
>
> *I2C is an addressable multi-drop bus. Each transaction starts with the STM32 transmitting a **START condition** followed by a **7-bit device address**:*
>
> ** **SSD1306 OLED**: Address `0x3C` (`0x78` shifted).*
>
> ** **DS1307 RTC**: Address `0x68` (`0xD0` shifted).*
>
> *Only the peripheral whose internal hardware address matches responds with an **ACK** bit. All other slaves on the bus ignore subsequent data bytes until the next repeated START or STOP condition.*

Q4: Why is the time stored in BCD (Binary-Coded Decimal) in the DS1307?

> ***Answer:***
>
> *Binary Coded Decimal allocates 4 bits (a nibble) per decimal digit ($0-9$). For example, minute 59 is represented as `0101 1001b` (`0x59`). This simplifies internal hardware display and date math on dedicated RTC silicon without needing division/modulus arithmetic. Our STM32 driver converts this to standard integers using:*

`BCD_TO_DEC(x) = (((x) >> 4) * 10) + ((x) & 0x0F)`.

Q5: What is the Clock Halt (CH) bit, and how does your firmware handle it?

Answer:

*Bit 7 of the DS1307 seconds register (`0x00`) is the **Clock Halt (CH)** flag. When power is first applied with a new battery, the oscillator is stopped by default (`CH = 1`) to preserve battery life during shipping. In our `DS1307_Init()` function, we read register `0x00`, clear bit 7 (`secReg &= 0x7F`), and write it back, ensuring the 32.768 kHz oscillator starts ticking immediately.*

---

3. OLED & Graphics Architecture

Q6: How does the pure C SSD1306 driver work without third-party graphics libraries?

Answer:

*A 1024-byte array `SSD1306_Buffer[128 * 64 / 8]` is maintained in STM32 SRAM as a **monochrome framebuffer**. Drawing functions (`DrawPixel`, `DrawLine`, `DrawRect`, `DrawString`) manipulate bits in this RAM buffer. When a redraw is requested, `ssd1306_UpdateScreen()` transmits the entire 1024 bytes across 8 pages over I2C to the SSD1306 controller in a fast burst transaction.*

Q7: Why did you not use direct I2C pixel drawing instead of a framebuffer?

Answer:

SSD1306 memory is organized in 8-bit vertical pages. You cannot set a single pixel directly without reading the existing 8 vertical pixels first (read-modify-write). Doing this over I2C for every line and character would cause severe bus traffic and screen flicker. The 1 KB SRAM framebuffer allows instant drawing with zero I2C traffic until the frame is fully composed.

---

4. Safety Logic & Human-Machine Interface (HMI)

Q8: How does the Smart Pill Box prevent medication errors?

Answer:

*The system implements a **two-stage verification state machine**:*

1. When an alarm triggers (e.g. Pill 1 at 07:30), the screen displays `"TAKE: >> PILL 1 <<"`, and LED 1 blinks with a 500 ms acoustic alert.

*2. If the user presses the wrong button (Button 2 instead of Button 1), the system switches to a rapid **150 ms warning alert**, flashing an inverse video `"WRONG PILL!"` screen instructing the user to stop and press the correct button.*

3. An alarm only clears and displays `"DOSE TAKEN"` when the matching correct button is pressed.

Q9: How are button bounces handled in your firmware?

Answer:

*Instead of using blocking delays (**HAL_Delay**), button inputs use a non-blocking edge-detection algorithm based on **HAL_GetTick()**. When a button pin transitions from HIGH to LOW, a state timestamp is checked:*

```
if (now - lastButtonPress > DEBOUNCE_TIME) { ... }
```

A 200 ms debounce threshold rejects mechanical contact bounce without stalling the clock display or alarm buzzer.

Q10: How does the on-device clock adjustment work using only 3 buttons?

Answer:

*1. **Button 3 (PB15)**: Opens the **SETUP_MENU**.*

*2. **Button 1 (PB12)**: Cycles through options: **1. PILL 1 TIME**, **2. PILL 2 TIME**, **3. ADJUST CLOCK**.*

*3. **Button 2 (PB14)**: Selects the active item.*

4. During editing:

** Button 1 increments the flashing field (Hours $0-23$, then Minutes $0-59$).*

** Button 2 advances to the next field and opens the Save Confirmation screen.*

*5. When confirmed, **DS1307_SetTime()** writes the new time directly to the RTC over I2C.*

---

5. Storage (MicroSD Card) & Diagnostic Architecture

Q11: How does the MicroSD Card communicate with the STM32, and on what pins?

Answer:

*The MicroSD module communicates via **hardware SPI1** on Port A pins:*

** **PA4**: Chip Select (**CS** - Active LOW GPIO Output)*

** **PA5**: Clock (**SCK** - SPI1 Alternate Function)*

** **PA6**: Master In Slave Out (**MISO** - SPI1 Input with Pull-Up)*

** **PA7**: Master Out Slave In (**MOSI** - SPI1 Alternate Function)*

*This cleanly preserves Port B for the I2C bus (**PB6**/**PB7**), push buttons (**PB12**, **PB14**, **PB15**), and indicator outputs (**PB0**, **PB1**, **PB10**).*

Q12: What exact files are stored on the SD Card and why?

Answer:

*1. **CONFIG.TXT**: Holds non-volatile configurations for **dose1** and **dose2** hours and minutes. Loaded at startup to restore schedules after power cycling, and overwritten whenever schedules are modified in the setup menu.*

*2. **LOGS.CSV**: An audit trail recording every medication event (**DOSE_1_TAKEN**, **DOSE_2_TAKEN**, **WRONG_BUTTON**, **CLOCK_ADJUST**, **CONFIG_UPDATE**) formatted with RTC timestamps. It can be opened directly on a PC in Microsoft Excel.*

Q13: What happens if the SD card or RTC is physically unplugged or absent?

Answer:

*The firmware uses **non-blocking graceful degradation**:*

** If the SD card is missing, **`SD_Init()`** fails and sets **`sdAvailable = false`**. The firmware defaults to internal RAM schedules and does not crash or block.*

** If the DS1307 RTC is missing or unpowered, the system falls back to **`HAL_GetTick()`** elapsed time accumulation, displaying an alert symbol (**`!`**) while keeping alarms, timers, and buttons fully responsive.*

** During power-on, a diagnostic screen runs an I2C scan (**`0x01`**–**`0x7F`**) and reports both **`RTC: OK/NONE`** and **`SD: OK/NONE`**.*

Q14: Can the DS1307 RTC be powered with 3.3V, and why?

Answer:

*No. The DS1307 datasheet specifies an operating voltage of $4.5\text{ V} \text{ to } 5.5\text{ V}$. Its internal power-fail threshold disables the I2C interface when $V_{CC} < 1.25 \times V_{BAT}$ ($\approx 3.75\text{ V}$), forcing battery-backup mode. Therefore, V_{CC} is connected to the Blue Pill's 5 V pin. The STM32's **`PB6`** and **`PB7`** pins are 5V-tolerant (FT) and communicate safely with the bus.*